Experiment No. 4

Title

JavaScript Form Validation Using ES6 Syntax

Aim

To implement client-side form validation using JavaScript ES6 features in order to enhance data accuracy and user experience.

Software Required

- Text Editor (Notepad / VS Code)

- Web Browser (Chrome / Firefox)

Theory

JavaScript is a client-side scripting language used to add interactivity and dynamic behavior to web pages. Client-side form validation is a technique used to verify user input before it is submitted to the server. This helps in reducing incorrect data entry, improving user experience, and minimizing unnecessary server-side processing.

Form validation ensures that all required fields are filled correctly and that the entered data follows the expected format, such as valid email addresses and non-empty input fields. Validation performed on the client side provides immediate feedback to the user.

ES6 (ECMAScript 2015) is an enhanced version of JavaScript that introduces features such as let, const, arrow functions, and improved event handling. These features make the validation code cleaner, more readable, and easier to maintain.

CSS is used along with JavaScript to visually indicate validation errors and success messages. Error messages are displayed near the input fields, improving usability and clarity for the user.

Basic JavaScript Validation Syntax

1. Event Handling (Form Submission):Used to handle form submission and prevent default behavior.

```
form.addEventListener("submit", (event) => {
   event.preventDefault();
});
```

2. Accessing Input Values:Used to read user-entered data from form fields.

```
const name = document.getElementById("name").value.trim();
const email = document.getElementById("email").value.trim();
```

3. Condition Checking:Used to verify whether input fields are empty or invalid.

```
if (name === "") {
   // display error message
}
```

4. Real-Time Validation (Input / Blur Events): Used to validate fields while the user is typing or when focus is lost.

```
nameInput.addEventListener("input",() => {
   if (nameInput.value.trim() === "") {
      nameError.textContent = "Name is required";
   }
 else {
      nameError.textContent = "";
   }
});
```

5. Enabling and Disabling Submit Button:Used to allow form submission only when all fields are valid.

```
submitBtn.disabled = true;

if (isFormValid) {
   submitBtn.disabled = false;
}
```

6. Dynamic CSS Class Manipulation:Used to visually indicate valid or invalid input fields.

```
input.classList.add("error-field");
input.classList.remove("success-field");
input.classList.add("success-field");
```

7. Character Counter for Textarea:Used to display live character count.’’’

```
textarea.addEventListener("input", () => {
   charCount.textContent = textarea.value.length + "/200";
});
```

8. Email Validation Using Regular Expression:Used to validate proper email format.

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailPattern.test(email)) {
   // invalid email
}
```

9. Displaying Error Messages in HTML:Used to show validation messages near input fields.

```
errorSpan.textContent = "Invalid input";
```

10. Clearing Error Messages:Used before validation to reset previous messages.

```
errorSpan.textContent = "";
```

11. Resetting Form After Successful Submission:Used to clear all input fields.

```
form.reset();
```

12. Storing Data in Local Storage:Used to save form data in the browser.

```
localStorage.setItem("username", name);
```

13. Retrieving Data from Local Storage:Used to access stored data when page reloads.

```
const storedName = localStorage.getItem("username");
```

14. Confirmation Dialog:Used to confirm user actions.

```
if (confirm("Are you sure?")) {
   form.reset();
}
```

Procedure

1. Create a new folder and save all files inside the same folder.

2. Create an HTML file containing a contact form.
3. Assign unique id attributes to all input fields.
4. Create an external CSS file to style the form and error messages.
5. Create a JavaScript file using ES6 syntax.
6. Link the CSS file using the <link> tag.
7. Link the JavaScript file using the <script> tag.
8. Add validation logic using JavaScript.
9. Prevent form submission if validation fails.
10. Display appropriate error or success messages.
11. Open the HTML file in a web browser and test the validation.

Program Code

1.HTML File (contact.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Contact Form</title>

   <!-- External CSS -->
   <link rel="stylesheet" href="Exp4style.css">
</head>
<body>
<h2>Contact Form</h2>

<form id="Exp4">
   Name:
   <input type="text" id="name" required>
   <span class="error" id="nameError"></span><br><br>
```

```
    Email:
    <input type="email" id="email" required>
    <span class="error" id="emailError"></span><br><br>

    Message:
    <textarea id="message" maxlength="150" ></textarea>
    <div id="count">0/150</div>
    <span class="error" id="messageError"></span><br><br>

    <input type="submit" value="Submit" id="submitBtn">
</form>
<p class="success" id="successMsg"></p>

<!-- External JavaScript -->
<script src="Exp4.js"></script>

</body>
</html>
```

2. CSS File (style.css)

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f6f9;
    padding: 30px;
}
form {
    background-color: white;
    padding: 20px;
    max-width: 400px;
    margin: auto;
    border-radius: 5px;
```

```
}
input, textarea {
    width: 100%;
    padding: 8px;
    margin-top: 5px;
    border: 1px solid #b5b5b5;
    border-radius: 4px;
}
input:focus, textarea:focus {
    outline: none;
    border-color: #2563eb;
}
input[type="submit"] {
    background-color: #2563eb;
    color: white;
    border: none;
    padding: 10px;
    cursor: pointer;
    margin-top: 10px;
}
input.valid {
    border-color: #1eff00;
    background-color: #f0fff6;
}
input.invalid {
    border-color: #ff0000;
    background-color: #ffffff;
}
input[type="submit"]:disabled {
    background-color: rgb(0, 0, 0);
    cursor: not-allowed;
```

```
}
.error {
    color: red;
    font-size: 14px;
}
.success {
    color: green;
    font-size: 16px;
    margin-top: 10px;
}
```

JavaScript File (script.js)

```
const form = document.getElementById("Exp4");

const nameInput = document.getElementById("name");
const emailInput = document.getElementById("email");
const messageInput = document.getElementById("message");
const successMsg = document.getElementById("successMsg");

const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
const messageError = document.getElementById("messageError");
const submitBtn = document.getElementById("submitBtn");

const charCount = document.getElementById("count");

// task 1
nameInput.addEventListener("input",() => {
    if (nameInput.value.trim() === "") {
        nameError.textContent = "Name is required";
    }
```

```
 else {
        nameError.textContent = "";
    }
    disable_submitBtn();
});
emailInput.addEventListener("input",() => {
    if (emailInput.value.trim() === "") {
        emailError.textContent = "Email is required";
    }
    else {
        emailError.textContent = "";
    }
    disable_submitBtn();
});
messageInput.addEventListener("input", () => {
    if(messageInput.value.trim() === "") {
        messageError.textContent = "Message is required";
    }
    else {
        messageError.textContent = "";
    }
    disable_submitBtn();
});

// task 2
messageInput.addEventListener("input", () => {
    charCount.textContent = messageInput.value.length + "/150";
    let len = messageInput.value.length;
    if (len == 150) {
        messageError.textContent = "Character limit reached";
    }
```

```
});

//task 4
nameInput.addEventListener("input", () => {
    if (nameInput.validity.valid) {
        nameInput.classList.add("valid");
        nameInput.classList.remove("invalid");
        nameError.textContent = "";
    }
    else {
        nameInput.classList.add("invalid");
        nameInput.classList.remove("valid");
    }
})
emailInput.addEventListener("input", () => {
    if (emailInput.validity.valid) {
        emailInput.classList.add("valid");
        emailInput.classList.remove("invalid");
        emailError.textContent = "";
    }
    else {
        emailInput.classList.add("invalid");
        emailInput.classList.remove("valid");
    }
})

//task 3
function disable_submitBtn() {
    const nameValid = nameInput.value.trim() != "";
    const emailValid = emailInput.value.trim() != "" || emailInput.validity.valid ;
    const messageValid = messageInput.value.trim() != "";
```

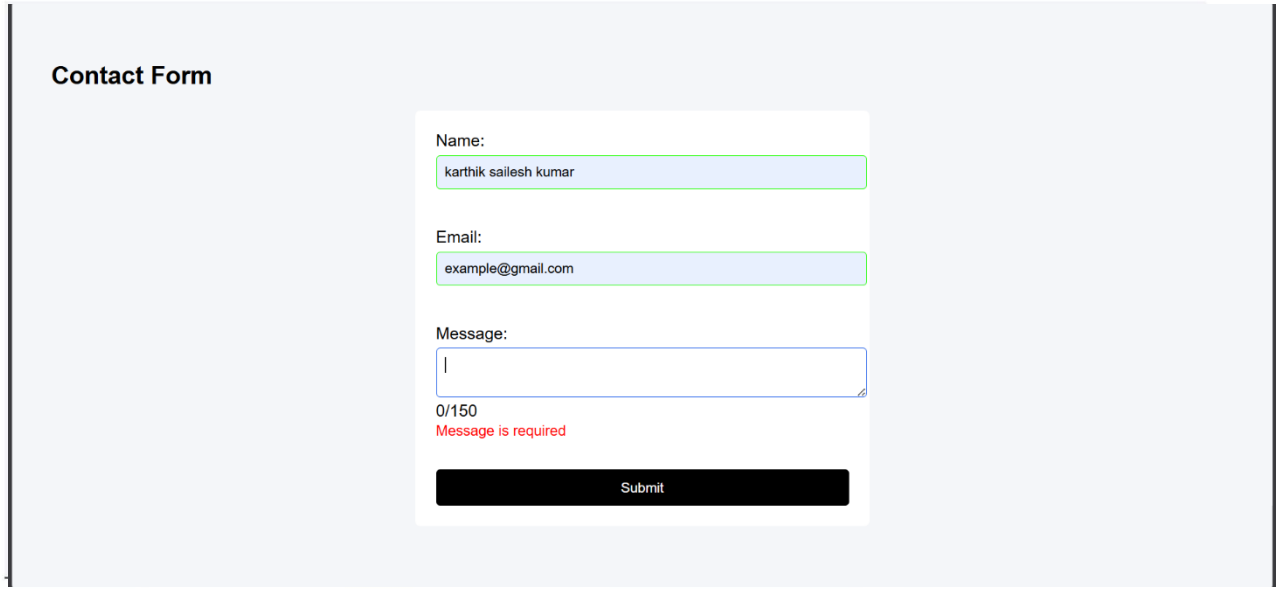
```
    submitBtn.disabled = !(nameValid && emailValid && messageValid);
}

//task 5
nameInput.addEventListener("input", () => {
  localStorage.setItem("name", nameInput.value);
});
const storedName = localStorage.getItem("name");
if (storedName) {
  nameInput.value = storedName;
}

emailInput.addEventListener("input", () => {
  localStorage.setItem("email", emailInput.value);
});
const storedEmail = localStorage.getItem("email");
if (storedEmail) {
  emailInput.value = storedEmail;
}

disable_submitBtn();

// submit
form.addEventListener("submit", (event) => {
    event.preventDefault();

    const name = document.getElementById("name").value.trim();
    const email = document.getElementById("email").value.trim();
    const message = document.getElementById("message").value.trim();
```

```
    document.getElementById("nameError").textContent = "";
    document.getElementById("emailError").textContent = "";
    document.getElementById("messageError").textContent = "";
    document.getElementById("successMsg").textContent = "";

    let isValid = true;

    if (name === "") {
        document.getElementById("nameError").textContent = "Name not filled";
        isValid = false;
    }
    if (email === "") {
        document.getElementById("emailError").textContent = "Email not filled";
        isValid = false;
    }
    else if (!email.includes("@")) {
        document.getElementById("emailError").textContent = "Invalid email format";
        isValid = false;
    }
    if (message === "") {
        document.getElementById("messageError").textContent = "Message not filled";
        isValid = false;
    }
    if (isValid) {
        document.getElementById("successMsg").textContent = "Form submitted successfully!";
        form.reset();
        charCount.textContent = "0/150";
        /* localStorage.removeItem("name");
        localStorage.removeItem("email"); */
    }
});
```

Student Tasks

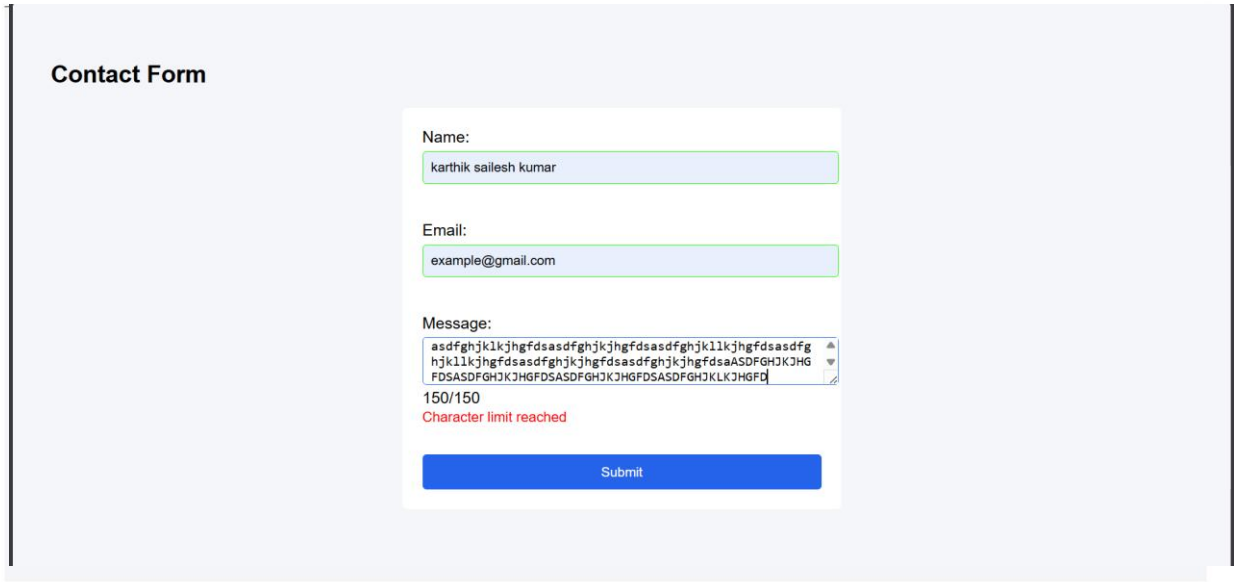
Task 1: Real-Time Validation (on input / blur)

Implement real-time validation so that error messages appear as the user types or leaves an input field, instead of only on form submission.



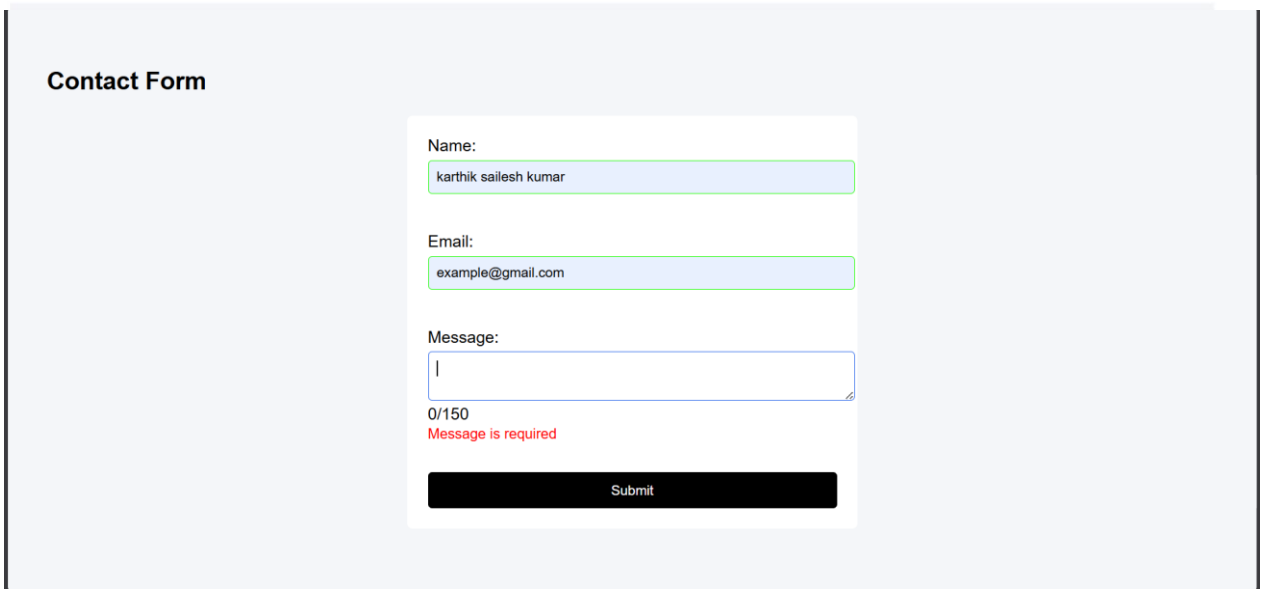
Task 2: Add Character Counter for Message Field

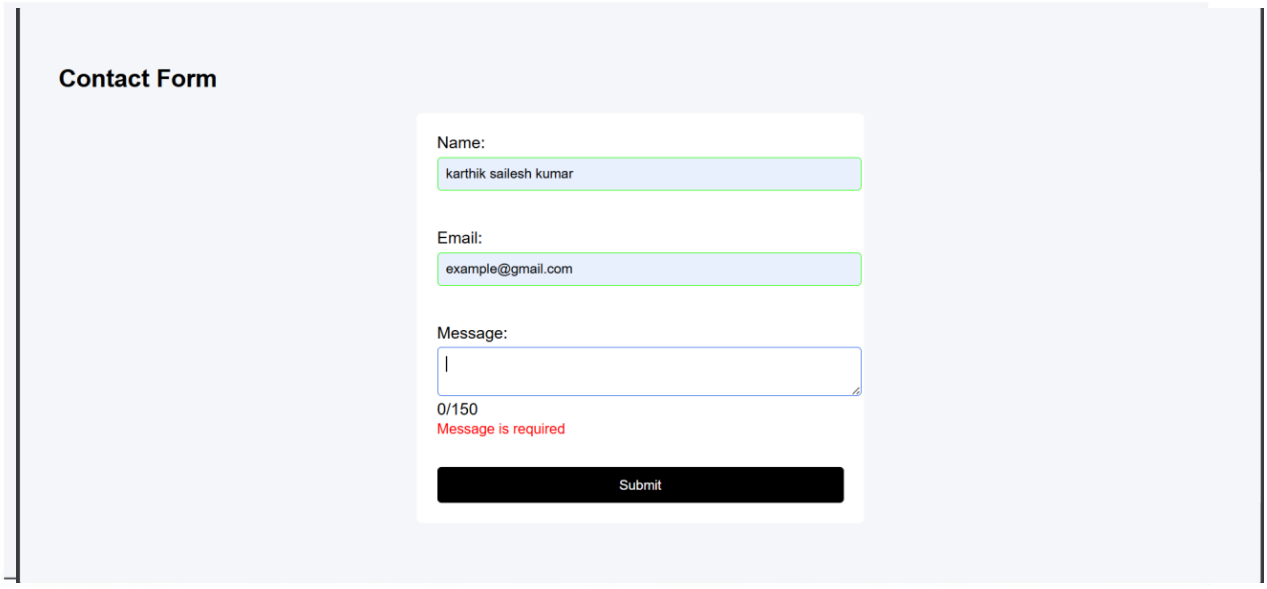
Display a live character count for the message textarea and restrict input to a maximum number of characters.



Task 3: Disable Submit Button Until Form Is Valid

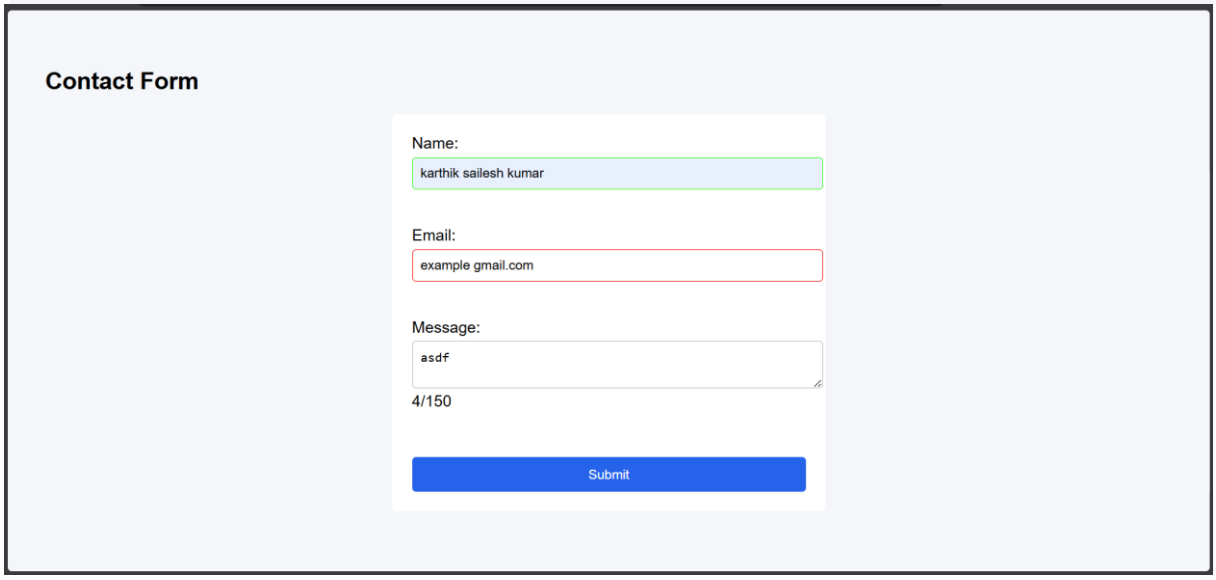
Disable the submit button by default and enable it only when all form fields contain valid data.





Task 4: Add Field Highlighting for errors

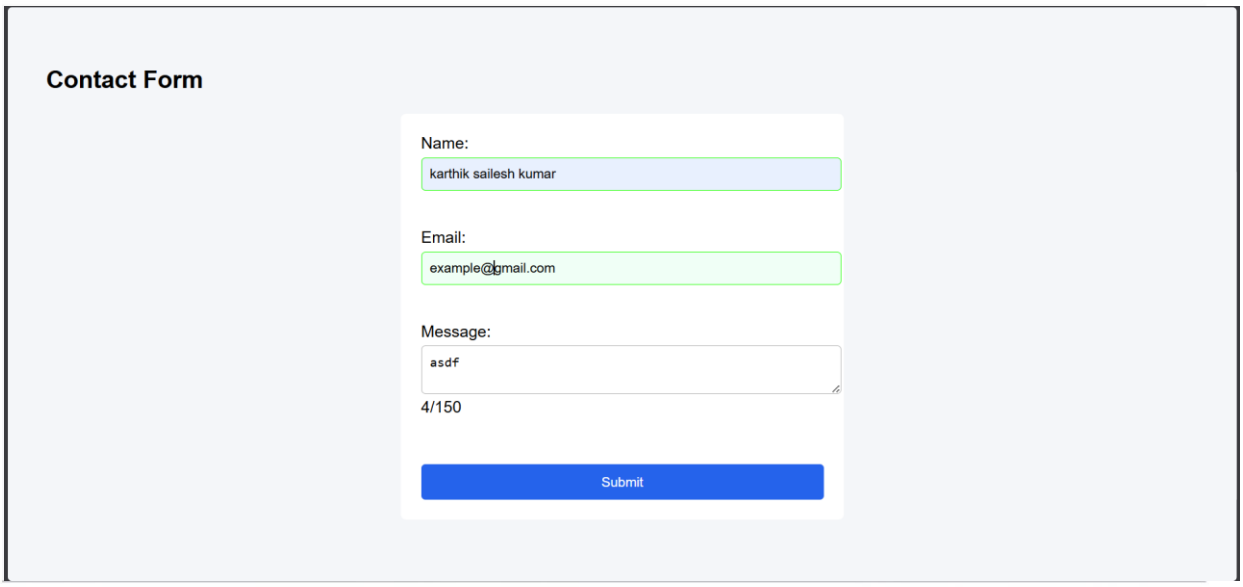
Change the border color of invalid input fields to red and valid fields to green using CSS classes controlled by JavaScript.



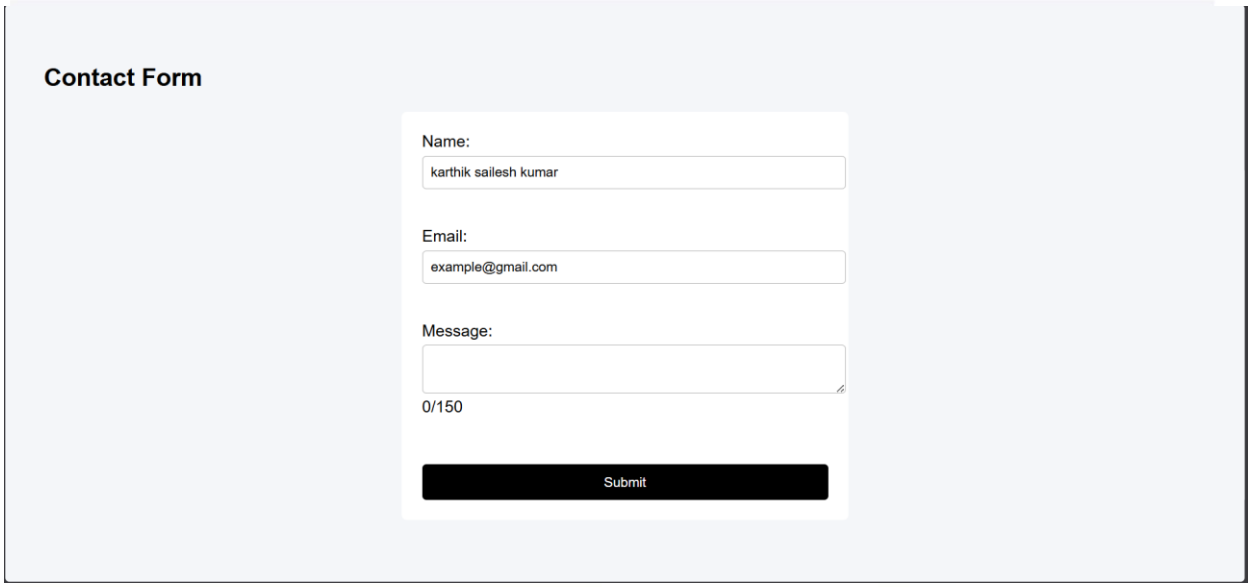
Task 5: Store Submitted Data in Browser Memory

Store the submitted form data in localStorage and display it when the page reloads.

Before Reload (Not Submitted) :



After Reload (Submitted) :



Result

Client-side form validation was successfully implemented using JavaScript ES6 features. CSS was used to display validation errors and success messages, ensuring accurate data input and improved user experience.